

SENG 326 Software Architecture Group Project
Task2: Target Architectural Styles
Refactoring Checkstyle: Metrics and Size Violation Checks

Target Style	Description	Compulsory	Not accepted
Strict Layered Architecture	A strict layered architecture with enforced downward dependencies only.	Define exactly four layers . Enforce compile-time layer rules Strong separation of concerns All metric and size checks reside in the same layer	Skip layers Allow cross-layer dependencies Hide logic in utility classes Conceptual layering
Service-Oriented Architecture (Orchestration-Based)	SOA with a central orchestrator controlling service execution.	Define exactly three coarse-grained services. Introduce a single orchestrator Services communicate only via the orchestrator Explicit workflow definition	Service-to-service calls Message buses Event-based coordination Microservices
Service-Oriented Architecture (Message Bus Based)	SOA using a central message bus for service integration and coordination.	Define services as independent, loosely coupled units Communicate only via the message bus Use explicit message contracts for coordination Include message routing and mediation logic Message-based coordination	Direct service-to-service calls Central orchestration or workflow engines Shared mutable state across services Microservices

Target Style	Description	Compulsory	Not accepted
Event-Driven Architecture	Architecture based on events and reactive components.	Define events Use an event bus Decouple publishers and subscribers Support asynchronous execution semantics Reaction-based coordination	Central orchestrators Direct component calls Shared state for coordination
Blackboard / Repository Architecture	A shared problem-state architecture with cooperating knowledge sources.	Introduce a central blackboard Implement knowledge sources that read/write the blackboard Include a controller for execution phases State-centric collaboration	Message passing for coordination Direct calls between checks Event buses
Pipe-and-Filter Architecture	A linear data-flow architecture with independent processing stages.	Define explicit pipes and filters Pass data explicitly between filters Ensure filter independence Data-flow-oriented processing Replaceable pipeline stages	Shared mutable state Other types of coordination Layered shortcuts
Hexagonal Architecture (Ports & Adapters)	A domain-centric architecture isolating business logic from infrastructure.	Define input and output ports Implement suitable adapters Keep domain free of infrastructure dependencies	Framework dependencies in the domain Layered dependency chains Direct IO in core logic Strict layered architecture